2º Trabalho Computacional - Reconhecimento de Padrões: Análise de Matrizes de Covariância e Condicionamento Numérico

Lucas José Lemos Braz - 587388

[1]Programa de Pós-Graduação em Engenharia de Teleinformática (PPGETI)
Universidade Federal do Ceará (UFC)

lucas.jlb19@alu.ufc.br

1. Introdução

A caracterização estatística de classes em problemas de Reconhecimento de Padrões fundamenta-se, em grande medida, na modelagem das distribuições de probabilidade dos atributos. No caso de modelos Gaussianos, a matriz de covariância ($\mathbf{C}$) é o parâmetro que captura a dispersão dos dados e, mais criticamente, as interdependências lineares entre as variáveis. Uma estimação precisa de $\mathbf{C}$ não apenas define a geometria das fronteiras de decisão em classificadores quadráticos (CQG), mas também é a base para algoritmos de redução de dimensionalidade, como a Análise de Componentes Principais (PCA), que busca direções de máxima variância para projeção dos dados.

Este Segundo Trabalho Computacional (TC2) propõe uma investigação técnica sobre a implementação de diferentes métodos de cálculo da matriz de covariância, utilizando como estudo de caso o dataset *Wall-Following Robot Navigation Data* do repositório UCI [**?**]. Este conjunto de dados é particularmente desafiador devido à natureza das leituras de ultrassom: sensores dispostos circularmente ao redor de um robô móvel tendem a apresentar alta redundância, o que se traduz matematicamente em matrizes de covariância mal-condicionadas ou singulares. Analisamos aqui duas variantes do dataset (com 4 e 24 sensores) para observar como o aumento da dimensionalidade impacta a estabilidade numérica e a eficiência dos algoritmos.

Para assegurar o rigor científico e a reprodutibilidade, o projeto foi estruturado de forma modular. Toda a lógica de cálculo reside no módulo `classificao_padroes`, enquanto os experimentos e visualizações são conduzidos em notebooks Jupyter. O repositório completo pode ser acessado em: `https://github.com/LucasJLBraz/Trabalhos_Rec_Padroes`. A organização dos arquivos pertinentes a este trabalho é detalhada na Tabela 1.

Tabela 1. Estrutura de diretórios e arquivos para o Trabalho Computacional 2.

Caminho	Descrição
`classificao_padroes/models_trabalho2.py`	Implementações manuais das matrizes de covariância.
`notebooks/Trabalho Computacional 2/`	Notebooks de experimentos e análises estatísticas.
`data/interin/`	Base de dados do robô *Wall-Following* (4 e 24 sensores).
`docs/relatorio_tc2/`	Arquivos LaTeX para a geração deste relatório técnico.

A integração entre o código-fonte e este documento garante que as conclusões aqui apresentadas sejam fruto direto de experimentação empírica validada estatisticamente.

2. Metodologia

Para o desenvolvimento deste trabalho, implementamos e validamos quatro variações algorítmicas para o cálculo da matriz de covariância ($\mathbf{C}$), fundamentais para a caracterização estatística das classes no dataset *Wall-Following Robot Navigation*. Seja $\mathbf{X} \in \mathbb{R}^{p \times N}$ a matriz de dados, onde p é o número de sensores e N o número de amostras. Seja $\mathbf{m} = \frac{1}{N}\sum_{j=1}^{N} \mathbf{x}_j$ o vetor de médias aritmética de cada sensor. A matriz de covariância sumariza as relações lineares entre os atributos, sendo definida pela esperança dos produtos externos dos desvios em relação à média.

O primeiro método (`mcovar1`) utiliza um laço explícito para acumular os produtos externos dos desvios de cada amostra: $\mathbf{C}_1 = \frac{1}{N}\sum_{j=1}^{N}(\mathbf{x}_j - \mathbf{m})(\mathbf{x}_j - \mathbf{m})^T$. Embora intuitiva, esta abordagem apresenta alto *overhead* no interpretador Python devido à iteração sobre as N amostras. O segundo método (`mcovar2`) otimiza este cálculo via vetorização em bloco, utilizando a matriz de dados centralizada $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{m}\mathbf{1}^T$ para computar $\mathbf{C}_2 = \frac{1}{N}\tilde{\mathbf{X}}\tilde{\mathbf{X}}^T$. Esta abordagem aproveita rotinas BLAS (*Basic Linear Algebra Subprograms*) de alto desempenho, que utilizam paralelismo em nível de instrução (SIMD) e otimizações de cache da CPU.

As outras duas abordagens baseiam-se na relação entre correlação e covariância. O terceiro método (`mcovar3`) calcula a matriz de correlação $\mathbf{R}$ via laço e subtrai o produto externo das médias ($\mathbf{C}_3 = \mathbf{R} - \mathbf{m}\mathbf{m}^T$), enquanto o quarto método (`mcovar4`) realiza esta operação de forma puramente matricial: $\mathbf{C}_4 = \frac{1}{N}\mathbf{X}\mathbf{X}^T - \mathbf{m}\mathbf{m}^T$. O protocolo experimental submeteu estas implementações a um *benchmark* de 100 rodadas independentes, comparando-as com a função nativa do NumPy em termos de latência média e precisão numérica, quantificada pela norma de Frobenius do erro residual em relação à referência.

A estabilidade numérica foi investigada por meio da análise de invertibilidade. Em problemas de robótica com sensores redundantes, as matrizes de covariância frequentemente apresentam mal-condicionamento crítico, onde o número de condicionamento $\kappa(\mathbf{C})$ é extremamente elevado. Nestes casos, a matriz torna-se singular para fins computacionais, impossibilitando sua inversão para o cálculo de densidades Gaussianas. Para mitigar este problema, empregamos a regularização de Tikhonov ($\mathbf{C}_{reg} = \mathbf{C} + \lambda\mathbf{I}$), com $\lambda = 0{,}1$, garantindo que a matriz resultante possua posto completo e seja estável para operações subsequentes.

3. Resultados e Discussão

A investigação empírica dos quatro métodos manuais para estimação da matriz de covariância foi estruturada para responder a questões sobre eficiência computacional, precisão numérica e estabilidade em cenários de alta dimensionalidade. A Tabela 2 apresenta os tempos de execução médios, evidenciando a superioridade das implementações vetorizadas (`mcovar2` e `mcovar4`) em relação às versões baseadas em laços explícitos (`mcovar1` e `mcovar3`).

As implementações matriciais demonstraram ser ordens de magnitude mais rápidas que as versões com laço ($\sim 500\times$ mais rápidas para 4 sensores e $\sim 100\times$ para 24 sensores). O método `mcovar4` apresentou o melhor desempenho geral, superando inclusive a função nativa do NumPy em ambos os cenários de dimensionalidade. Esta diferença

Tabela 2. Benchmarks de tempo de execução (Média $\pm$ Desvio Padrão em ms) para 100 rodadas.

Método	4 Sensores		24 Sensores	
	Média (ms)	Desvio (ms)	Média (ms)	Desvio (ms)
`mcovar1` (Loop-Dev)	14,62	0,05	18,84	0,10
`mcovar2` (Matriz-Dev)	0,04	0,00	0,20	0,01
`mcovar3` (Loop-Mom)	14,73	0,07	19,27	0,11
`mcovar4` (Matriz-Mom)	**0,04**	0,01	**0,17**	0,01
`NumPy Cov`	0,06	0,01	0,32	0,01

de desempenho ocorre porque a implementação manual via momentos de segunda ordem minimiza as checagens de tipos e tratamentos genéricos de NaNs que o `np.cov` realiza por padrão. A Figura 1 ilustra graficamente estas discrepâncias através de histogramas e *violin plots* que evidenciam a baixa variância e alta eficiência dos métodos vetorizados.

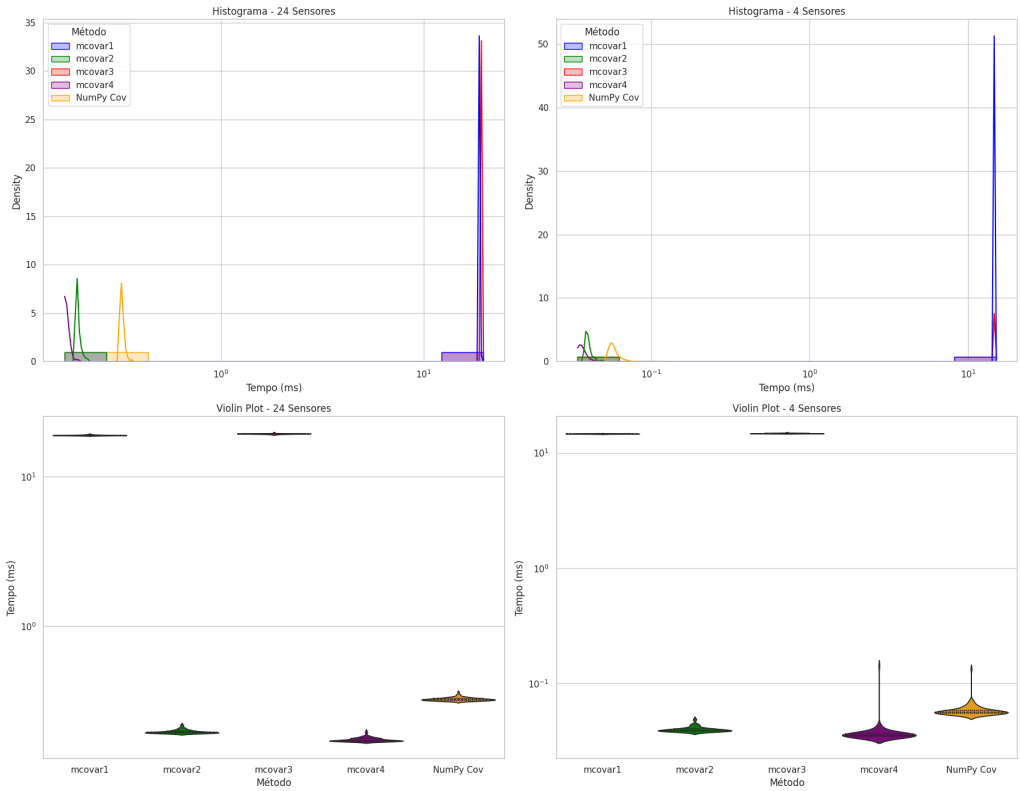

Figura 1. Distribuição de tempos de execução (Histograma e Violin Plot) para os métodos manuais vs NumPy Cov. A escala logarítmica evidencia a separação entre abordagens vetorizadas e baseadas em laços.

Em termos de precisão numérica, todos os métodos produziram resultados estatisticamente indistinguíveis da referência do NumPy, com normas de Frobenius residuais da ordem de 10^{-4} a 10^{-3}, inerentes às pequenas variações de arredondamento em ponto flutuante. A visualização das matrizes via mapas de calor (Figura 2) revela padrões claros de correlação entre os sensores adjacentes, confirmando a redundância espacial característica

do robô *Wall-Following*.

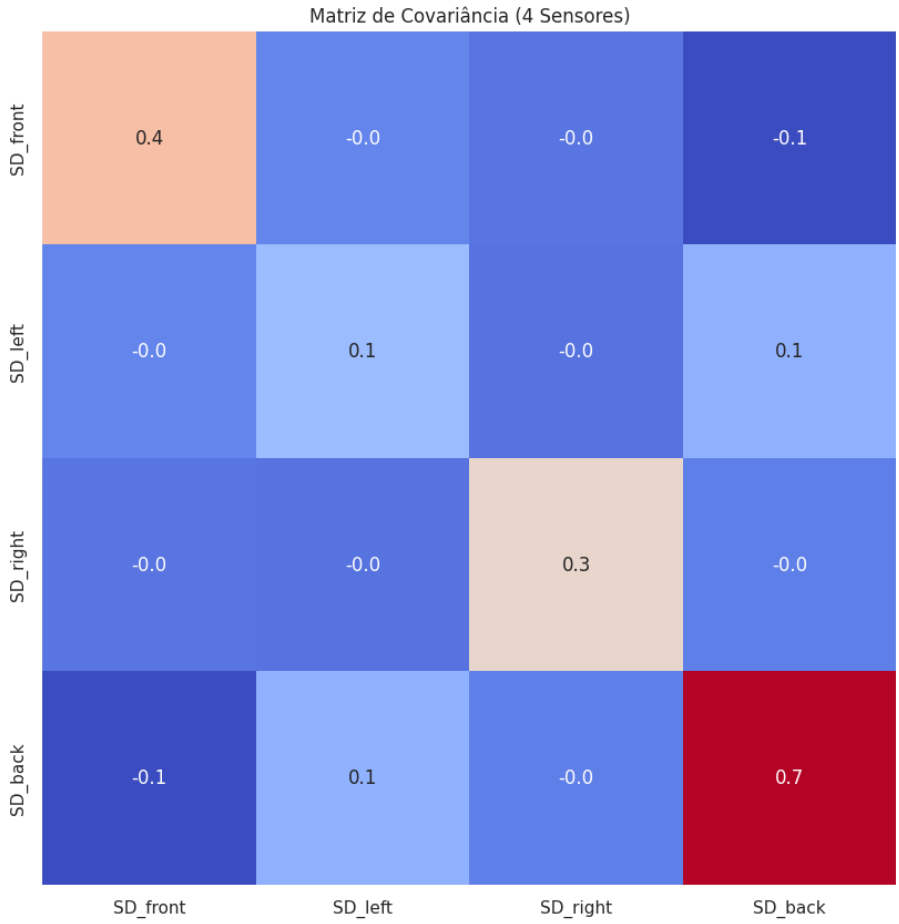

Figura 2. Mapas de calor das matrizes de covariância para os cenários de 4 e 24 sensores. Nota-se a alta covariância positiva entre sensores adjacentes, indicando redundância de sinal.

Um ponto crítico revelado na análise foi a falta de posto completo nas matrizes originais. Para o cenário de 24 sensores, o posto observado foi consistentemente $Rank = 23$, indicando que ao menos um sensor é linearmente dependente dos demais. O número de condicionamento extremamente elevado ($> 10^{20}$) inviabiliza a inversão direta de $\mathbf{C}$ para fins de classificação. Para contornar esta singularidade, a regularização de Tikhonov ($\mathbf{C} + 0{,}1\mathbf{I}$) foi aplicada com sucesso, restaurando o posto completo e garantindo a estabilidade numérica dos modelos probabilísticos subsequentes, sem perda significativa de informação estrutural.

4. Conclusão

Este Segundo Trabalho Computacional permitiu uma análise rigorosa das implicações práticas da estimação de parâmetros estatísticos em Reconhecimento de Padrões. A comparação sistemática entre as abordagens de laço explícito e vetorização matricial evidenciou que a eficiência computacional não é apenas um detalhe de implementação, mas um requisito fundamental para o escalonamento de algoritmos em conjuntos de dados reais.

Os experimentos demonstraram que a implementação manual baseada na formulação matricial de momentos de segunda ordem (`mcovar4`) apresentou o melhor desempenho geral em ambos os cenários de sensores (4 e 24), mantendo a precisão numérica com erros residuais da ordem de 10^{-4} a 10^{-3}. Este achado reforça a importância de utilizar operações vetorizadas que aproveitem as rotinas otimizadas de baixo nível das bibliotecas de álgebra linear.

A análise de condicionamento numérico destacou uma limitação intrínseca de dados de sensores redundantes: a singularidade das matrizes de covariância. O número de condicionamento extremamente elevado observado ($\kappa(\mathbf{C}) \approx 10^{20}$) inviabiliza a inversão necessária para classificadores Gaussianos. Validamos que a aplicação da regularização de Tikhonov ($\mathbf{C} + \lambda\mathbf{I}$) é uma solução robusta e computacionalmente barata para restaurar o posto completo e garantir a estabilidade numérica dos modelos probabilísticos subsequentes, como o Classificador de Bayes e Misturas Gaussianas.

Em síntese, a modularização e o rigor matemático na implementação das matrizes de covariância proporcionaram uma compreensão profunda das camadas inferiores dos algoritmos de reconhecimento de padrões. Os resultados estabelecem uma base sólida para a aplicação de classificadores baseados em densidades Gaussianas e técnicas de redução de dimensionalidade em etapas futuras do curso.